



HoneyComb: A Flexible LLM-Based Agent System for Materials Science

Anonymous ACL submission

Abstract

The emergence of specialized large language models (LLMs) has shown promise in addressing complex tasks for materials science. Many LLMs, however, often struggle with distinct complexities of material science tasks, such as materials science computational tasks, and often rely heavily on outdated implicit knowledge, leading to inaccuracies and hallucinations. To address these challenges, we introduce *HoneyComb*, the first LLM-based agent system specifically designed for materials science. *HoneyComb* leverages a novel, high-quality materials science knowledge base (MatSciKB) and a sophisticated tool hub (ToolHub) to enhance its reasoning and computational capabilities tailored to materials science. MatSciKB is a curated, structured knowledge collection based on reliable literature, while ToolHub employs an Inductive Tool Construction method to generate, decompose, and refine API tools for materials science. Additionally, *HoneyComb* leverages a retriever module that adaptively selects the appropriate knowledge source or tools for specific tasks, thereby ensuring accuracy and relevance. Our results demonstrate that *HoneyComb* significantly outperforms baseline models across various tasks in materials science, effectively bridging the gap between current LLM capabilities and the specialized needs of this domain. Furthermore, our adaptable framework can be easily extended to other scientific domains, highlighting its potential for broad applicability in advancing scientific research and applications.

1 Introduction

The emergence of large language models (LLMs) (OpenAI, 2024; Anthropic, 2024; Touvron et al., 2023b,a) in recent years has brought about the application of LLMs across a wide range of fields related to science and engineering (AI4Science and Quantum, 2023). This has resulted in a number of new benchmarks measuring the capabilities of

language models to perform scientific tasks (Wang et al., 2023; Sun et al., 2024; Mirza et al., 2024; Song et al., 2023a) along with the development of custom LLMs and LLM-based systems for scientific domains including chemistry (Bran et al., 2023; Boiko et al., 2023), biology (Madani et al., 2023) and materials science (Song et al., 2023b; Gupta et al., 2022; Walker et al., 2021).

While much progress has been made in adapting LLMs to common tasks in natural language processing (Song et al., 2023a,b), many more challenges remain in having LLMs be effective agents for real-world materials science tasks (Miret and Krishnan, 2024; Miret et al., 2024). As highlighted by Zaki et al. (2023), LLMs often fail in performing important computational tasks for materials science. Common mistakes by most LLMs include conceptual errors where models fail to retrieve correct concepts, equations, or facts relevant to the questions, and factual hallucinations where incorrect information is generated. An analysis by Miret and Krishnan (2024) also revealed that LLMs by themselves struggle to generate relevant and correct information pertaining to specialized materials science tasks. While Song et al. (2023b) showed that instruction fine-tuning can help in improving performance, the high costs of continuous model training and fine-tuning make retraining-based approaches challenging to scale. This is further compounded by the fact that relevant knowledge is continuously updated through a diversity of knowledge sources, including pre-print servers (e.g., arXiv and ChemRxiv), peer-reviewed literature, open encyclopedias (e.g. Wikipedia) and relevant websites. Furthermore, prior work has shown that utilizing external tools may be a more promising approach to solve complex scientific tasks instead of relying entirely on an LLMs internal knowledge (Zheng et al., 2024; Buehler, 2024a). To jointly address these challenges, we propose transforming LLMs into LLM-based agents that access external knowl-

edge and tools to boost their performance. This approach has already shown promise in adjacent domains, such as chemistry (Bran et al., 2023; Boiko et al., 2023) by enabling the the models to access real-time data and utilize computational as well as domain-specific tools. Altogether, the LLM-based agents showcase greater capabilities and performance compared to their native LLM counterparts.

In this paper, we present **HoneyComb**, the first, to the best of our knowledge, LLM-based agent system specifically designed for materials science. While there has been emerging research in LLMs for scientific domains, few studies have focused on developing comprehensive agent systems for materials science. Our work addresses two critical challenges: First, **MatSciKB** alleviates the challenge of obtaining reliable and relevant professional knowledge for materials. As such, MatSciKB ensures the agent has access to the most current and accurate information is essential for effective performance. Second, **Tool-Hub** provides materials science specific tools to augment the agent’s capabilities. These tools enable the agent to perform specialized computational tasks and enhance its overall functionality. As detailed in Section 4, we observe that with the aid of MatSciKB and Tool-Hub, HoneyComb outperforms its native LLM counterparts in a more reliable manner given its ability to utilize up-to-date knowledge and tools.

2 Background

2.1 LLMs for Material Science

Advancements in text mining and information extraction from scientific publications have significantly benefited the application of LLMs for materials science (Kononova et al., 2021; Swain and Cole, 2016). Early work include the development of specialized BERT models (Devlin et al., 2018), such as MatSciBERT (Gupta et al., 2022) and MatBERT (Walker et al., 2021). Song et al. (2023b) and Xie et al. (2023) leveraged instruction fine-tuning to develop a LLaMa-based (Touvron et al., 2023a) tailored to materials science that matched the capabilities of commercial LLMs at the time of publication. The emergence of powerful commercial LLMs (OpenAI, 2024; Anthropic, 2024) has further expanded the possibility of applying LLMs to materials science. Yet, commercial LLMs remain expensive, opaque in their methodology with consistent errors and shortcomings (Zaki et al., 2023; Miret and Krishnan, 2024), and open-source

LLMs for materials science remain sparse. This motivates the need for a practical LLM-based system that is useful for real-world materials science tasks.

Given this need, we propose HoneyComb as an open-source system to augment the capabilities of diverse LLMs. HoneyComb integrates specialized tools as well as a dynamic retrieval system to enhance the functionality any LLMs specifically for material science. By leveraging relevant knowledge source through MatSciKB and auxilliary tools through Tool-Hub, HoneyComb manages to improve the accuracy and relevance of the outputs of LLMs for materials science, while also addressing common challenges associated with static LLM applications in dynamic research fields.

2.2 Tool-Based LLM Agents for Scientific Applications

Prior work has shown success in expanding the capabilities of LLMs by augmenting their capabilities with diverse sets of tools (Qin et al., 2023b,a; Chern et al., 2023; Wang et al., 2024). Many works rely on pre-built integration frameworks, such as LangChain (Topsakal and Akinci, 2023), to build the relevant interfaces between the LLMs and the desired capabilities, such as search engine APIs. Wang et al. (2024) provides a recent survey of common approaches, challenges and applications of tool-based LLMs and their applications to various technology and scientific fields.

One major application of tool-based LLMs is in query processing and optimization, where agents evaluate initial search results and iteratively refine queries to increase relevance and accuracy (Buehler, 2024a,b). This approach addresses the limitations of isolated LLMs, which may struggle to handle ambiguous query contexts. In generating structured datasets for solar cell materials, agents gather pertinent information from a vast array of scientific papers to automate data input and synthesis (Xie et al., 2024; Liu et al., 2024). Furthermore, agents can utilize various tools to help answer specific questions by tapping into external resources (Cheng et al., 2024). For example, ChemCrow by Bran et al. (2023) integrates 18 expert-designed tools, such as literature search, molecule modification, and reaction execution, to autonomously execute chemical syntheses. Tool augmentation has also shown success in other research in the chemistry domain to enable real-world experiments using LLMs (Yoshikawa et al., 2023; Jablonka

et al., 2023; Boiko et al., 2023). Coscientist by Boiko et al. (2023), for examples, relies on specialized tools to extend the capabilities of GPT4 and thereby invoke domain-specific functionalities that are not inherently present within the LLM alone. The success of agent-based approaches in adjacent domains motivates the creation of HoneyComb that extends the capabilities of LLMs specifically for materials science.

3 HoneyComb

In this work, we introduce HoneyComb, shown in Figure 1, a specialized agent system designed to advance materials science research. It integrates three key components: 1) *MatSciKB*, a comprehensive knowledge base; 2) *ToolHub*, which includes general tools for accessing up-to-date information broadly and specialized tools developed through an Inductive Tool Construction method for targeted material science queries; 3) *Retriever*, utilizing a hybrid approach for efficient and precise information retrieval.

3.1 MatSciKB

Our MatSciKB knowledge base integrates a diverse array of sources, as detailed in Table 1. This collection is meticulously curated to include material science papers from ArXiv, relevant Wikipedia entries, textbooks, comprehensive datasets, pertinent mathematical formulas, and concrete GPT-generated examples tailored to material science. Each information source is thoroughly described in Appendix A.

The architectural framework of MatSciKB is thoughtfully structured into 16 distinct categories pertinent to material science. These are detailed in Appendix C and are organized in a tree-like structure. MatSciKB supports efficient searching and CRUD (Create, Read, Update, Delete) operations (Giannaros et al., 2023), which are vital for both the application and ongoing maintenance of the database. Given the continuously evolving and expanding body of knowledge in the materials science domain, capabilities for efficient updates and searches based on real-time information are crucial for research and engineering applications. Additionally, our structured data approach enhances the integration of the diverse data sources commonly encountered in materials science (Miret and Krishnan, 2024). This structure not only facilitates easy access and management but also allows for seam-

less extension to include additional data modalities.

MatSciKB	
# Total Number of Data Entries	38,469
# Material Science Papers on Arxiv	20,384
# Wikipedia for Material Science	3,620
# Material Science Textbook	1,930
# Material Science Dataset	10,473
# Material Science Formula	57
# GPT-generated Examples	2,005

Table 1: Statistics of the MatSciKB knowledge base

3.2 Tool-Hub

The Tool-Hub in HoneyComb is bifurcated into *General Tools* and *Material Science Tools*. Both categories are organized through a unified interface that supports allow HoneyComb to make effective use of all available tools. **General Tools** provide researchers with access to the latest information filling gaps not covered by the static entries in MatSciKB. **Material Science Tools** are specifically designed to handle complex calculations and in-depth analyses. The details of the unified interface are further elucidated in Appendix D.

General Tools Construction

In materials science, one of the persistent challenges is keeping research outputs aligned with the diverse and ever-evolving data modalities that describe complex material systems (Miret and Krishnan, 2024). The diversity of data sources and measurements leads to a rapid evolution of knowledge in this field, necessitating tools that can effectively access and integrate recent findings. Traditional static databases, while useful, often lag in capturing the newest research, creating gaps that can impede the currency and relevance of scientific analysis in real-time. Further, the need to efficiently process complex and dynamic computational tasks within the research workflow remains inadequately addressed, often requiring manual intervention which can introduce errors and inefficiencies. Thus, constructing tools that can handle varying data modalities and complexities, and that can adapt to the continual advancements in materials science, is essential for advancing the field.

To address these challenges, HoneyComb has been designed with innovative solutions that markedly enhance research capabilities in materials science. First, we integrated General Tools that provide direct access to current publications and

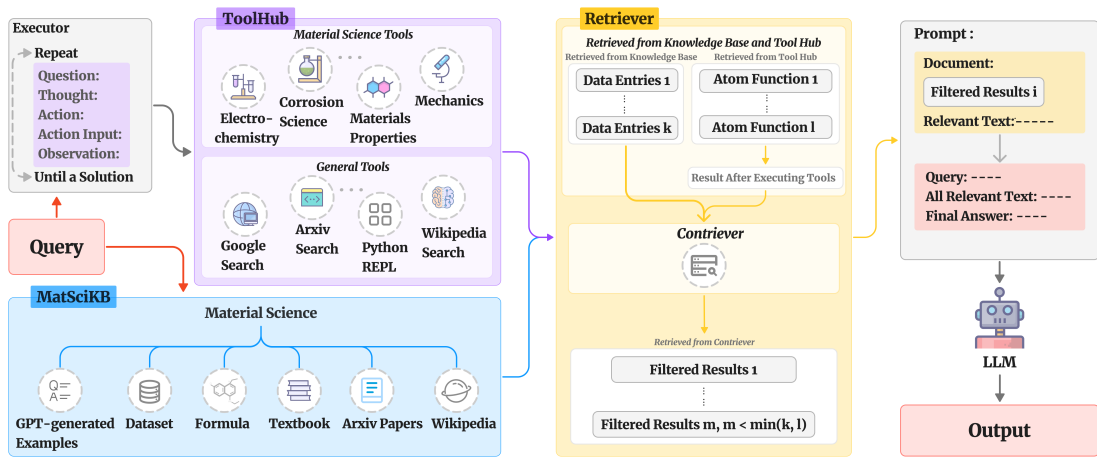


Figure 1: The overall architecture of HoneyComb. The model initiates with a query input that activates the knowledge retrieval phase, where pertinent data entries and atom function are extracted from the MatSciKB and Tool-Hub respectively. The Executor iterative calls the relevant tools from the tool hub, evaluating and refining these calls until a solution that adequately solves the query emerges. The preliminary solution generated by these tools is combined with relevant data entries, and then undergoes further processing by the Retriever. Finally, the Retriever consolidates and filters these input, ultimately feeding them into the LLM for final answer generation.

facilitate dynamic discussions, as shown in Table 2, effectively complementing the static MatSciKB. Secondly, recognizing the limitations of large language models (LLMs) in performing computational tasks, we implemented a Python REPL environment within HoneyComb. This environment is strategically utilized by the system when the agent, interacting with the Tool-Hub, identifies a need for basic numerical computations. The agent dynamically writes Python code for these tasks and executes it through the Python REPL, bypassing the LLM’s computational limitations. This automation not only streamlines data processing but also enhances the precision and reliability of numerical analyses in research activities.

General Tools	6
Google Search	
Google Scholar Search	
Arxiv Search	
Wikipedia Search	
YouTube Search	
Python REPL	

Table 2: HoneyComb-ToolHub

Inductive Tool Construction for Materials Science

Constructing domain-specific tool APIs presents

Algorithm 1 Inductive Tool Construction

Require: Train Set D_{train} , LLM M

Ensure: Set of atom tools A

- 1: $A \leftarrow \emptyset$ {Initialize the set of atom functions}
- 2: **for** each question q_i in D_{train} **do**
- 3: $f_i \leftarrow M(q_i)$ {Generate specific function for q_i }
- 4: Human verifies f_i
- 5: Decompose f_i into atom functions a_i
- 6: $A \leftarrow A \cup a_i$ {Add atom functions to the set}
- 7: **end for**
- 8: **return** A

significant challenges. It requires domain expert knowledge, and there are limited existing resources to draw upon. Additionally, many valuable data and tools are not open source, limiting their accessibility. Developing these tools is essential for effectively addressing the unique and complex queries inherent to materials science. The scarcity of pre-existing, specialized computational tools necessitates a methodical approach to tool construction and refinement.

We propose the *Inductive Tool Construction* method, delineated in Algorithm 1 for domain-specific tool APIs construction. It adopts a systematic approach to fabricate and refine computational tools specifically designed for material sci-

ence queries. The process initiates by selecting a random subset of computational questions from dataset D , designated as D_{train} for training, with the residual questions forming D_{test} . For each question $q_i \in D_{train}$, a designated LLM, M (such as GPT-4), is tasked to generate a Python function f_i that addresses q_i . After creation, each function f_i undergoes rigorous human verification to confirm its correctness.

However, the above procedures cannot ensure the generalizability of the constructed tool APIs. Thus, in the post-validation stage, we further use M to decompose each f_i into fundamental, reusable components known as atomic function a_i , which are crafted for extensive applicability across diverse queries, a detailed example is illustrated in Appendix F

3.3 Agent-ToolHub Interactions

In HoneyComb, interactions between the agent and the Tool Hub are governed by a meticulously structured two-phase decision-making protocol, specifically designed to address the challenges associated with efficiently handling large datasets.

1. Preliminary Tool Assessor: During the initial phase, the Assessor critically evaluates the incoming query to intelligently select a manageable subset of tools from the extensive suite available. This strategic selection is crucial as it prevents data overload at the decision-making stage, thereby ensuring that the Executor is not hindered by an excessive volume of data.

2. Strategic Tool Executor: As illustrated in Figure 2, the Executor then conducts a thorough evaluation of the selected tools. This involves a deliberate thought process aimed at determining the most effective course of action, specifically which tool to employ and how to precisely define the necessary inputs—whether they involve parameters for specialized tools or search terms for search engines. This tailored process meticulously meets each tool’s specific requirements.

The execution phase entails deploying the selected action, monitoring the outcomes, and iteratively refining the strategy until the response adequately resolves the initial query.

By structuring the interactions between the agent and the Tool Hub in this manner, we effectively address the primary issue of information overload, significantly enhancing the framework’s efficiency and effectiveness.

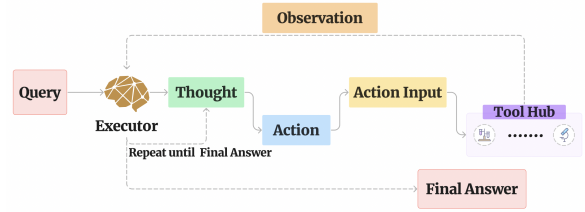


Figure 2: Tool executor interaction cycle in the HoneyComb system. The executor receives and integrates information from relevant tools (actions) and knowledge bases (thoughts) to iteratively arrive at a final answer for a given query.

3.4 Retriever

In this section, we present the retriever in HoneyComb which returns relevant texts or tools from MatSciKB and Tool Hub when a specific contexts is given. The retriever integrates both BM25 (Trotman et al., 2014) and Contriever (Izacard et al., 2022) algorithms, leveraging their respective strengths to achieve optimal information retrieval performance.

Specifically, the retriever employs a two-step strategy. Initially, BM25 utilizes efficient calculations of term frequency and inverse document frequency to rapidly process short text queries and keyword searches within long documents. The primary advantage of BM25 lies in its computational simplicity and rapid response, allowing HoneyComb to extract the 10 most relevant knowledge points from an extensive materials science knowledge base, ensuring exceptional speed and efficiency. This approach enables the provision of basic relevance matching results in a minimal timeframe.

Subsequently, we employs a pre-trained deep learning models (i.e. Contriever) to generate embedding vectors and compute their similarity, facilitating the understanding of complex linguistic structures and semantic information. The strength of Contriever resides in its capability to comprehend and process intricate language structures, contextual information, and semantic relationships, thereby delivering more precise and comprehensive retrieval results. Although Contriever operates at a slower pace compared to BM25, it pulls the most relevant results from the knowledge base and memory, as well as from tools invoked through the Tool-Hub, extracting the top 3 results. Its ability to precisely handle complex queries and diverse documents ensures high accuracy and relevance.

By combining BM25 and Contriever, our model

adeptly responds to simple queries with speed while offering enhanced accuracy and relevance for complex queries. This hybrid approach ensures that the model is both efficient and capable of addressing sophisticated query requirements, thereby providing comprehensive, efficient, and precise information retrieval services.

4 Experiments

We conduct experiments on two question answering datasets, namely MaScQA (Zaki et al., 2023) and SciQA (Johannes Welbl, 2017), to investigating the ability of HoneyComb in materials science tasks.

MaScQA, derived from the Graduate Aptitude Test in Engineering (GATE) in India, is tailored to reflect the real-world complexity and variety of issues encountered in material science. This highly competitive examination assesses a comprehensive understanding of various undergraduate subjects (Indian Institute of Technology Kanpur, 2023; Zaki et al., 2023). With its 650 questions covering 14 domains such as thermodynamics, atomic structure, and mechanical behavior, the dataset showcases a wide range of question types, from Multiple Choice Questions (MCQs), Numerical Answer Type (NUM), and Matching Type (MATCH) to MCQs with numerical options (MCQN). Specifically designed for advanced problem-solving, this dataset is crucial for ensuring that our ToolHub functions effectively in actual material science research and applications. It demonstrates the HoneyComb framework’s efficacy and adaptability in tackling complex material science issues within realistic scenarios. The second dataset, SciQA, comprises 11,679 multiple-choice questions that span the core disciplines of fundamental sciences from a variety of crowdsourced science exams (Johannes Welbl, 2017). This compilation not only underlines the dataset’s comprehensive and interdisciplinary nature but also focuses on fostering a nuanced conceptual understanding. SciQA serves as a critical testbed to ascertain whether the HoneyComb framework can augment the LLM’s capabilities beyond its initial programming. By integrating supplementary information, it aids in addressing intricate queries and unraveling complex scientific concepts that may have been overlooked during the initial training phase of the LLM. By bridging real-world complexities with rigorous academic standards, these datasets ensure that our MatSciKB

and ToolHub are not only versatile but also remain at the forefront of technological and scientific application.

The choice of models for our experiments was driven by the need to evaluate the HoneyComb framework’s enhancement capabilities across a spectrum of large language models known for their robust performance in diverse applications. We selected GPT-3.5, GPT-4 (OpenAI, 2024), LLaMA-2 (Touvron et al., 2023b), and LLaMA-3 (AI@Meta, 2024) due to their widespread use and proven effectiveness in handling complex language tasks. These models, with LLaMA-2 and LLaMA-3 having parameter sizes of 7 billion and 8 billion respectively, represent the current state-of-the-art in generalized language understanding and provide a solid baseline for benchmarking. Additionally, we included HoneyBee (Song et al., 2023b), a specialized model with a parameter size of 7 billion, tailored specifically for materials science. The inclusion of both general-purpose and specialized models allows us to showcase how domain-specific adaptations through HoneyComb can elevate a model’s functional scope beyond its original configuration, thus highlighting the adaptability and effectiveness of our framework.

4.1 HoneyComb Evaluation

We evaluated the performance of various models on MaScQA and SciQA, including HoneyBee, GPT-3.5, GPT-4, Llama2, and Llama3, and demonstrated the effects of using the HoneyComb. The results are illustrated in Table 3

The experimental results show that all models based on HoneyComb achieved significant improvements in accuracy on both MaScQA and SciQA. Specifically, on the MaScQA dataset, models such as HoneyBee and GPT-4 experienced substantial improvements, with HoneyBee’s accuracy improving by 16.76% and GPT-4’s by 20.61%. Other models also showed notable enhancements, with improvements ranging from 4.92 to 14.16%. On the SciQA dataset, the HoneyBee model saw a dramatic increase in performance, representing a huge improvement of 45.73%. HoneyComb based on GPT-3.5 and Llama3 showed more modest enhancements of around 0.14% to 0.32%, whereas HoneyComb based on GPT-4 and Llama2 experienced considerable improvements of approximately 5.70% and 2.87%, respectively.

Dataset	HoneyBee	HoneyBee 🐝	GPT-3.5	GPT-3.5 + 🐝	GPT-4	GPT-4 + 🐝	Llama2	Llama2 + 🐝	Llama3	Llama3 + 🐝
MaScQA	16.62	33.38	33.54	38.46	58.46	79.07	22.15	36.31	24.62	47.23
SciQA	33.96	79.69	90.69	90.83	90.84	96.54	75.79	78.66	93.00	93.32

Table 3: HoneyComb evaluation with diverse LLMs including open-source LLMs (HoneyBee (Song et al., 2023b), LLaMa2 (Touvron et al., 2023b), LLaMa3 (AI@Meta, 2024)) and commercial LLMs (GPT3.5, GPT4 (OpenAI, 2024)). The results show that HoneyComb consistently improves the performance of all LLMs for SciQA and MaScQA.

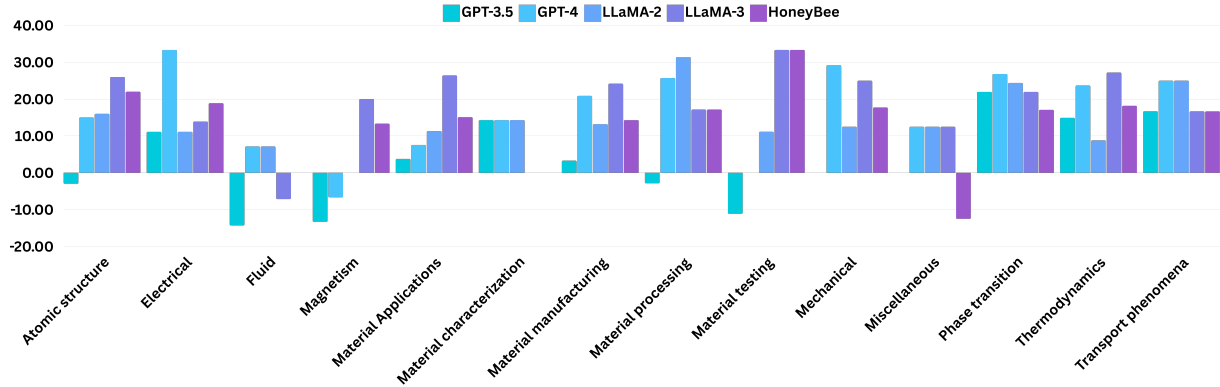


Figure 3: Improvements of various LLMs integrated with HoneyComb compared to relevant baseline LLMs for different materials science tasks. With few exceptions, HoneyComb improves the performance of all LLMs across all tasks showing the utility of tool augmentation.

4.2 HoneyComb Evaluation on MaScQA

We assess the performance improvements when integrating the HoneyComb framework with various large language models across predefined topics within the MaScQA dataset, as shown in Figure 3. The overall trend indicates that HoneyComb substantially enhances model performance. LLaMA-3 and HoneyBee exhibit impressive gains, particularly in 'Material Testing' where improvements of 33.34 percentage points are observed, showcasing HoneyComb's capability to effectively augment models with its advanced Tool-Hub and extensive MatSciKB.

However, GPT-3.5 displays a unique trend with declines across multiple topics including Atomic Structure, Fluid, Magnetism, Material Processing, and Material Testing. Despite having a higher baseline accuracy than LLaMA-3, LLaMA-2, and HoneyBee, GPT-3.5's performance dips more frequently when integrated with HoneyComb. This could be attributed to its training data's scope and depth, which, while extensive, may not align as effectively with HoneyComb's highly specialized material science enhancements. The sophisticated computational demands and the dynamic nature of materials science queries may expose limitations in GPT-3.5's ability to adapt its pre-existing knowl-

edge to the specific enhancements HoneyComb offers. This nuanced understanding highlights the importance of model and tool compatibility in achieving effective enhancements across diverse materials science domains, thereby informing further development and optimization of HoneyComb to ensure comprehensive and reliable support in all areas of materials science research.

4.3 Ablation Study

To study how each component of HoneyComb contributes to the overall performance, we conducted ablation studies in this section. We tested the performance of HoneyComb when retrieved only from MatSciKB or only from Tool Hub, respectively. We also report results without retriever, in such situation there is no way for MatSciKB and ToolHub results to be fed into the model. Experimental results are reported in Table 4.

The experimental results show that the best performance is achieved when both MatSciKB and ToolHub are used as reliable material knowledge references. HoneyComb improved the correctness on MaScQA and SciQA by 0.76% and 10.99% when compared to retrieving only from MatSciKB, and improved the correctness on MaScQA and SciQA by 5.84% and 0.22% when compared to retrieving only from Tool Hub. Therefore, we rec-

Table 4: Ablation Study Results for MaScQA and SciQA based on GPT-4

Benchmark	MatSciKB	ToolHub	Retriever	Accuracy
MaScQA				61.38
		✓	✓	73.23
	✓		✓	78.31
	✓	✓	✓	79.07
SciQA				90.84
		✓	✓	96.34
	✓		✓	85.57
	✓	✓	✓	96.56

ommend that users retrieve HoneyComb from both sources together when deploying or using it.

5 Conclusion

In this work, we introduced HoneyComb, a pioneering LLM-based agent system tailored for materials science. HoneyComb integrates a meticulously curated materials science knowledge base (MatSciKB) and a dual-layered ToolHub of general and specialized computational tools. It combines three critical components: MatSciKB, an inductively constructed ToolHub, and a precision-focused Retriever module. This ensures HoneyComb provides accurate, up-to-date information and performs complex computational tasks reliably.

Experimental results show that HoneyComb outperforms contemporary general-purpose models (e.g. GPT and LLaMa series) and specialized models (e.g. HoneyBee) in materials science QA tasks. HoneyComb effectively bridges the gap between advanced large language models and the specific needs of materials science research, exemplifying how specialized agent systems can advance scientific research and serve as a blueprint for future advancements in other knowledge-intensive fields.

Limitations

While HoneyComb significantly enhances the performance of current state-of-the-art models in various materials science QA tasks, there are limitations to its generalizability and applicability beyond the specific datasets and tasks it was trained on. Materials science is a diverse and intricate field, and it remains unclear how well HoneyComb would perform on tasks outside the MaScQA and SciQA benchmarks, particularly for more complex and novel challenges in materials science. Such challenges may include designing synthesis recipes for new materials or predicting material properties.

Additionally, HoneyComb’s reliance on high-quality LLMs for the knowledge base, tool construction, and retrieval processes can be a limitation. The performance of these components is contingent on the availability and capability of the underlying LLMs, which themselves may have inherent limitations. Furthermore, our work has primarily focused on the materials science domain, and further studies are required to evaluate how applicable and effective HoneyComb would be in other scientific fields.

Broader Impacts

By expanding the HoneyComb agent system, HoneyComb has the potential to accelerate scientific discovery and innovation, contributing to a deeper understanding of complex materials systems. This could not only lead to advancements in materials design, development, and application but also promote the discovery and optimization of new materials, benefiting a wide range of industries. Additionally, the versatility and adaptability of HoneyComb enable it to tackle challenges across various scientific domains, further broadening its scope and impact.

Our research does not raise major ethical concerns.

References

Microsoft Research AI4Science and Microsoft Azure Quantum. 2023. [The Impact of Large Language Models on Scientific Discovery: a Preliminary Study using GPT-4](#). *arXiv preprint arXiv:2311.07361*.

AI@Meta. 2024. [Llama 3 model card](#).

Anthropic. 2024. [Calude3](#).

Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. 2023. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578.

Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. 2023. [Chemcrow: Augmenting large-language models with chemistry tools](#). *Preprint*, arXiv:2304.05376.

Markus J. Buehler. 2024a. [Generative retrieval-augmented ontologic graph and multiagent strategies for interpretive large language model-based materials design](#). *ACS Engineering Au*, 4(2):241–277.

Markus J Buehler. 2024b. Mechgpt, a language-based strategy for mechanics and materials modeling that connects knowledge across scales, disciplines, and modalities. *Applied Mechanics Reviews*, 76(2):021001.

637	Yuheng Cheng, Ceyao Zhang, Zhengwen Zhang, Xi-	691
638	angrui Meng, Sirui Hong, Wenhao Li, Zihao Wang,	692
639	Zekai Wang, Feng Yin, Junhua Zhao, and Xiuqiang	693
640	He. 2024. Exploring large language model based in-	694
641	telligent agents: Definitions, methods, and prospects.	695
642	<i>Preprint</i> , arXiv:2401.03428.	
643	I-Chun Chern, Steffi Chern, Shiqi Chen, Weizhe Yuan,	696
644	Kehua Feng, Chunting Zhou, Junxian He, Graham	697
645	Neubig, Pengfei Liu, et al. 2023. Factool: Factu-	698
646	ality detection in generative ai—a tool augmented	699
647	framework for multi-task and multi-domain scenar-	700
648	ios. <i>arXiv preprint arXiv:2307.13528</i> .	701
649	Jacob Devlin, Ming-Wei Chang, Kenton Lee, and	702
650	Kristina Toutanova. 2018. Bert: Pre-training of deep	703
651	bidirectional transformers for language understand-	704
652	ing. <i>arXiv preprint arXiv:1810.04805</i> .	705
653	Anastasios Giannaros, Aristeidis Karras, Leonidas	706
654	Theodorakopoulos, Christos Karras, Panagiotis Kra-	707
655	nias, Nikolaos Schizas, Gerasimos Kalogeratos, and	708
656	Dimitrios Tsolis. 2023. Autonomous vehicles: So-	709
657	phisticated attacks, safety issues, challenges, open	710
658	topics, blockchain, and future directions. <i>Journal of</i>	
659	<i>Cybersecurity and Privacy</i> , 3(3):493–543.	
660	Maarten Grootendorst. 2022. Bertopic: Neural	711
661	topic modeling with a class-based tf-idf procedure.	712
662	<i>Preprint</i> , arXiv:2203.05794.	713
663	Tanishq Gupta, Mohd Zaki, NM Krishnan, et al. 2022.	714
664	Matscibert: A materials domain language model for	715
665	text mining and information extraction. <i>npj Computa-</i>	716
666	<i>tional Materials</i> , 8(1):1–11.	
667	Indian Institute of Technology Kanpur. 2023. Gate	717
668	2023: Graduate aptitude test in engineering. Ac-	
669	cessed: 2024-06-14.	
670	Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebas-	718
671	tian Riedel, Piotr Bojanowski, Armand Joulin, and	719
672	Edouard Grave. 2022. Unsupervised dense informa-	720
673	tion retrieval with contrastive learning. <i>Transactions</i>	721
674	<i>on Machine Learning Research</i> .	722
675	Kevin Maik Jablonka, Qianxiang Ai, Alexander Al-	723
676	Feghali, Shruti Badhwar, Joshua D Bocarsly, An-	724
677	dres M Bran, Stefan Bringuier, L Catherine Brinson,	725
678	Kamal Choudhary, Defne Circi, et al. 2023. 14 exam-	726
679	ples of how llms can transform materials science and	727
680	chemistry: a reflection on a large language model	728
681	hackathon. <i>Digital Discovery</i> , 2(5):1233–1250.	729
682	Matt Gardner Johannes Welbl, Nelson F. Liu. 2017.	730
683	Crowdsourcing multiple choice science questions.	731
684	Olga Kononova, Tanjin He, Haoyan Huo, Amalie Tre-	732
685	wartha, Elsa A Olivetti, and Gerbrand Ceder. 2021.	733
686	Opportunities and challenges of text mining in materi-	734
687	als research. <i>Iscience</i> , 24(3).	
688	LangChain contributors. 2023. Langchain: Open-	735
689	source library for building language-based agents.	736
690	Online; accessed 17-June-2023.	737
	Yue Liu, Sin Kit Lo, Qinghua Lu, Liming Zhu, De-	738
	hai Zhao, Xiwei Xu, Stefan Harrer, and Jon Whittle.	739
	2024. Agent design pattern catalogue: A collection	740
	of architectural patterns for foundation model based	741
	agents. <i>Preprint</i> , arXiv:2405.10467.	742
	Ali Madani, Ben Krause, Eric R. Greene, Subu Subrama-	743
	nian, Benjamin P. Mohr, James M. Holton, Jose Luis	744
	Olmos, Caiming Xiong, Zachary Z. Sun, Richard	745
	Socher, James S. Fraser, and Nikhil Naik. 2023.	746
	Large language models generate functional protein	
	sequences across diverse families. <i>Nat. Biotechnol.</i> ,	
	41(8):1099–1106.	
	Santiago Miret and NM Krishnan. 2024. Are llms ready	
	for real-world materials discovery? <i>arXiv preprint</i>	
	<i>arXiv:2402.05200</i> .	
	Santiago Miret, NM Anoop Krishnan, Benjamin	
	Sanchez-Lengeling, Marta Skreta, Vineeth Venu-	
	gopal, and Jennifer N Wei. 2024. Perspective on	
	ai for accelerated materials design at the ai4mat-2023	
	workshop at neurips 2023. <i>Digital Discovery</i> .	
	Adrian Mirza, Nawaf Alampara, Sreekanth Kunchapu,	
	Benedict Emoekabu, Aswanth Krishnan, Mara Wil-	
	helmi, Macjonathan Okereke, Julianne Eberhardt,	
	Amir Mohammad Elahi, Maximilian Greiner, et al.	
	2024. Are large language models superhuman	
	chemists? <i>arXiv preprint arXiv:2404.01475</i> .	
	OpenAI. 2024. Openai . Accessed: 2024-06-14.	
	Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen,	
	Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang,	
	Chaojun Xiao, Chi Han, Yi Ren Fung, Yusheng Su,	
	Huadong Wang, Cheng Qian, Runchu Tian, Kunlun	
	Zhu, Shihao Liang, Xingyu Shen, Bokai Xu, Zhen	
	Zhang, Yining Ye, Bowen Li, Ziwei Tang, Jing Yi,	
	Yuzhang Zhu, Zhenning Dai, Lan Yan, Xin Cong,	
	Yaxi Lu, Weilin Zhao, Yuxiang Huang, Junxi Yan,	
	Xu Han, Xian Sun, Dahai Li, Jason Phang, Cheng	
	Yang, Tongshuang Wu, Heng Ji, Zhiyuan Liu, and	
	Maosong Sun. 2023a. Tool learning with foundation	
	models. <i>Preprint</i> , arXiv:2304.08354.	
	Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan	
	Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang,	
	Bill Qian, et al. 2023b. Toolllm: Facilitating large	
	language models to master 16000+ real-world apis.	
	<i>arXiv preprint arXiv:2307.16789</i> .	
	Yu Song, Santiago Miret, and Bang Liu. 2023a. MatSci-	
	NLP: Evaluating scientific language models on ma-	
	terials science language tasks using text-to-schema	
	modeling. In <i>Proceedings of the 61st Annual Meet-</i>	
	<i>ing of the Association for Computational Linguistics</i>	
	<i>(Volume 1: Long Papers)</i> , pages 3621–3639, Toronto,	
	Canada. Association for Computational Linguistics.	
	Yu Song, Santiago Miret, Huan Zhang, and Bang Liu.	
	2023b. Honeybee: Progressive instruction finetuning	
	of large language models for materials science. In	
	<i>Findings of the Association for Computational Lin-</i>	
	<i>guistics: EMNLP 2023</i> , pages 5724–5739.	

747	Liangtai Sun, Yang Han, Zihan Zhao, Da Ma, Zhennan Shen, Baocai Chen, Lu Chen, and Kai Yu. 2024. Sci-eval: A multi-level large language model evaluation benchmark for scientific research. In <i>Proceedings of the AAAI Conference on Artificial Intelligence</i> , volume 38, pages 19053–19061.	805
748		806
749		
750		
751		
752		
753	Matthew C Swain and Jacqueline M Cole. 2016. Chem-dataextractor: a toolkit for automated extraction of chemical information from the scientific literature. <i>Journal of chemical information and modeling</i> , 56(10):1894–1904.	
754		
755		
756		
757		
758	Oguzhan Topsakal and Tahir Cetin Akinici. 2023. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In <i>International Conference on Applied Engineering and Natural Sciences</i> , volume 1, pages 1050–1056.	
759		
760		
761		
762		
763	Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. Llama: Open and efficient foundation language models . <i>Preprint</i> , arXiv:2302.13971.	
764		
765		
766		
767		
768		
769		
770	Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023b. Llama 2: Open foundation and fine-tuned chat models . <i>Preprint</i> , arXiv:2307.09288.	
771		
772		
773		
774		
775		
776		
777		
778		
779		
780		
781		
782		
783		
784		
785		
786		
787		
788		
789		
790		
791		
792		
793	Andrew Trotman, Antti Puurula, and Blake Burgess. 2014. Improvements to bm25 and language models examined. In <i>Proceedings of the 19th Australasian Document Computing Symposium</i> , pages 58–65.	
794		
795		
796		
797	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention is all you need . <i>Preprint</i> , arXiv:1706.03762.	
798		
799		
800		
801	Nicholas Walker, Amalie Trewartha, Haoyan Huo, Sanghoon Lee, Kevin Cruse, John Dagdelen, Alexander Dunn, Kristin Persson, Gerbrand Ceder, and Anubhav Jain. 2021. The impact of domain-specific	
802		
803		
804		
	pre-training on named entity recognition tasks in materials science. <i>Available at SSRN 3950755</i> .	806
	Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. <i>Frontiers of Computer Science</i> , 18(6):186345.	807
		808
		809
		810
		811
	Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. 2023. Scibench: Evaluating college-level scientific problem-solving abilities of large language models. <i>arXiv preprint arXiv:2307.10635</i> .	812
		813
		814
		815
		816
		817
	Tong Xie, Yuwei Wan, Wei Huang, Zhenyu Yin, Yixuan Liu, Shaozhou Wang, Qingyuan Linghu, Chunyu Kit, Clara Grazian, Wenjie Zhang, et al. 2023. Darwin series: Domain specific large language models for natural science. <i>arXiv preprint arXiv:2308.13565</i> .	818
		819
		820
		821
		822
	Tong Xie, Yuwei Wan, Yufei Zhou, Wei Huang, Yixuan Liu, Qingyuan Linghu, Shaozhou Wang, Chunyu Kit, Clara Grazian, Wenjie Zhang, and Bram Hoex. 2024. Creation of a structured solar cell material dataset and performance prediction using large language models . <i>Patterns</i> , 5(5).	823
		824
		825
		826
		827
		828
	Naruki Yoshikawa, Marta Skreta, Kourosh Darvish, Sebastian Arellano-Rubach, Zhi Ji, Lasse Bjørn Kristensen, Andrew Zou Li, Yuchi Zhao, Haoping Xu, Artur Kuramshin, et al. 2023. Large language models for chemistry robotics. <i>Autonomous Robots</i> , 47(8):1057–1086.	829
		830
		831
		832
		833
		834
	Mohd Zaki, Jayadeva, Mausam, and N. M. Anoop Krishnan. 2023. Mascqa: A question answering dataset for investigating materials science knowledge of large language models . <i>Preprint</i> , arXiv:2308.09115.	835
		836
		837
		838
	Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V Le, and Denny Zhou. 2024. Take a step back: Evoking reasoning via abstraction in large language models . <i>Preprint</i> , arXiv:2310.06117.	839
		840
		841
		842
		843
	Appendix	844
	A MatSciKB Knowledge Source	845
	• ArXiv Paper	846
	– Included all papers indexed under the “material science” keyword on ArXiv.	847
		848
	– Data entries structured into key-value pairs: key is the paper title, and value is the abstract.	849
		850
		851
	– Data Entries Count: 20,384	852
	• Wikipedia Material Science Concepts	853
	– Scraped all 438 pages categorized under “Materials Science” on Wikipedia.	854
		855

856	– Each section within a page was separated	902
857	as a distinct data entry.	903
858	– Content formulated into key-value pairs,	904
859	with keys as section titles and values as	905
860	content.	906
861	– Data Entries Count: 3,620	907
862	• Material Science Textbook	908
863	– Sourced 6 publicly available textbooks.	909
864	– Converted each textbook PDF file to text	
865	documents.	
866	– Broke each textbook into data entries by	
867	each section in a chapter.	
868	– Formulated data entries into key-value	
869	pairs, with keys as section titles and val-	
870	ues as content.	
871	– Data Entries Count: 1,930	
872	• Material Science Dataset	
873	– Utilized the multiple-choice dataset	
874	SciQA.	
875	– Extracted "support" column from the	
876	dataset that provides background knowl-	
877	edge for each question.	
878	– Each extracted "support" is treated as a	
879	data entry, with keys as the knowledge	
880	piece and values as empty strings, em-	
881	phasizing their concise and standalone	
882	nature.	
883	– Data Entries Count: 10,473	
884	• Material Science Formula	
885	– Formulas collected from Wikipedia’s	
886	dedicated pages for material science for-	
887	mulas.	
888	– Each formula is stored as key-value pair	
889	in the database, where the key represents	
890	the name of the formula and the value	
891	contains the formula equation itself.	
892	– Data Entries Count: 57	
893	• GPT-generated Examples	
894	– Used a specific prompt to generate 50	
895	material science questions at a time, out-	
896	put in CSV format along with a confi-	
897	dence score. Please refer to Appendix B	
898	for the detailed prompt.	
899	– Human reviewers then selected questions	
900	with higher confidence scores for inclu-	
901	sion in the dataset.	
	– Inspiration for question types was drawn	902
	from an external resource offering a wide	903
	range of material science questions and	904
	answers.	905
	– The key-value pairs were structured with	906
	questions as the keys and answers as the	907
	values.	908
	– Data Entries Count: 2,005	909
	B Prompt for GPT-Generated Examples	910
	Please generate 50 instances of material science	911
	questions, specifically atomic structure and in-	912
	teratomic bonding, in a CSV format in the fol-	913
	lowing order: question, answer, accuracy, con-	914
	fidence_score - accuracy: for factual questions,	915
	please evaluate the answer by comparing it with	916
	known facts. this field should be a number be-	917
	tween 0 and 1. - confidence_score: how confident	918
	are you with the answer. this field should be a	919
	number between 0 and 1. - Here are sample in-	920
	stances without accuracy and confidence_score:	921
	“In terms of which of the following properties,	922
	metals are better than ceramics?”, “ductility”	923
	“In the wave-mechanical model of an atom, what do	924
	degenerate energy levels have?”, “equal energy”	925
	“Which of the following molecules is diamag-	926
	netic?”, “CO” - Examples of generated instances: -	927
	“What is the valence electron configuration of car-	928
	bon?”, “2s ² 2p ² ”, 0.95, 0.85 - “What type of crystal de-	929
	fect occurs when there is a line of irregularity in the	930
	lattice structure?”, “dislocation defect”, 0.96, 0.91	931
	C Tree-Structure MatSciKB	932
	MatSciKB is organized as a hierarchical tree with	933
	the parent node “Material Science” branching into	934
	16 child nodes representing specific domains within	935
	materials science. Below is a simplified representa-	936
	tion of this structure:	937
	{	938
	"Material Science": {	939
	"Children": {	940
	"Thermodynamics": {"Children": {"KB_1":	941
	{"KB_2": {}, "KB_3": {}},	942
	"Atomic Structure": {"Children": {"KB_4":	943
	{"KB_5": {}, "KB_6": {}},	944
	...	945
	"Miscellaneous": {"Children": {"KB_7":	946
	{"KB_n": {}, "KB_{n+1}": {}}}	947
	} }	948
	Each child node encompasses knowledge base	949
	(KB) data entries relevant to its category. In the	950

construction of MatSciKB, we predefined 16 topics that align with core areas in materials science. They are 'Miscellaneous', 'Material testing', 'Fluid', 'Material characterization', 'Magnetism', 'Transport phenomena', 'Material processing', 'Electrical', 'Phase transition', 'Material Applications', 'Material manufacturing', 'Mechanical', 'Atomic structure', 'Thermodynamics', "Formula", "Fundamental_Science_Knowledge"]

To categorize the data entries within these nodes, we utilized BertTopic, a state-of-the-art topic modeling tool based on transformers and c-TF-IDF, which automatically identifies and clusters documents with high granularity and contextual relevance (Vaswani et al., 2023; Grootendorst, 2022). The integration of BertTopic allowed for the dynamic clustering of MatSciKB entries into 16 predetermined categories.

The process involved the following steps:

1. **Initial Clustering:** BertTopic was applied to cluster all data entries into more than the target number of categories, based on the textual content of each entry.
2. **Cluster Analysis and Selection:** Human reviewers analyzed each cluster, identifying those whose common keywords and themes closely aligned with one of the predefined 16 topics.
3. **Category Assignment:** Entries from clusters that aligned well with a predefined topic were assigned to that category, and then removed from the dataset.
4. **Iterative Refinement:** The remaining entries underwent subsequent rounds of clustering and analysis. This process was repeated until no entries were left unclassified.

D Tools Unified Interface Using LangChain

LangChain is an advanced framework designed to enhance applications that utilize LLM by offering standardized interfaces for various modules (LangChain contributors, 2023). This framework facilitates the seamless integration and efficient management of LLM with external tools and systems. Utilizing LangChain, HoneyComb has developed a unified interface that standardizes the integration of a wide array of tools.

In HoneyComb, the unified interface provided by LangChain ensures that all tools, regardless of their specific function, are treated as standardized LangChain objects. This standardization is achieved by defining each tool with a consistent set of attributes:

1. **Function Signature:** Each tool is defined with a clear function signature that specifies input and output types,
2. **Metadata Description:** Each tool is accompanied by metadata that describes its purpose, suitable use cases, parameters description.

Examples of function signatures and metadata descriptions in HoneyComb are:

• Google Search

- **Function Signature:**
Google_Search(query: str, timeout: Optional[int] = 30) -> str
- **Metadata Description:** General web search for up-to-date information across various topics.

• Wikipedia Search

- **Function Signature:**
Wikipedia_Search(topic: str, summarize: bool = True) -> str
- **Metadata Description:** Retrieves and optionally summarizes detailed Wikipedia articles, particularly useful for quick reference checks.

• A Sample Mass Flow Rate Tool

- **Function Signature:**
calculate_initial_mass_flow_rate(args: str) -> float
- **Metadata Description:** See figure 4.

E Knowledge Base Prompts

E.0.1 Material Science Papers on Arxiv

Arxiv hosts a vast collection of the latest research papers in the field of material science, reflecting the current frontiers and developmental trends. Most papers on Arxiv are freely accessible, reducing the cost and barriers to acquiring the latest knowledge. Covering various branches and interdisciplinary areas of material science, Arxiv papers provide

Calculate the initial mass flow rate of liquid metal draining from a cylindrical vessel through a nozzle.

Parameters:

- args (str): A string containing the required parameters separated by "|" in the following order:
 - density (float): Density of the liquid metal in kg/m³
 - nozzle_diameter (float): Diameter of the nozzle in mm
 - discharge_coefficient (float): Discharge coefficient of the nozzle (dimensionless)
 - height (float): Height of the liquid metal column in the vessel in meters

Returns:

- float: Initial mass flow rate in kg/s.

Figure 4: Metadata Description of a Sample Mass Flow Rate Tool

a comprehensive understanding of the field’s research landscape. Additionally, the rapid publication of research findings on Arxiv shortens the time from research to dissemination, facilitating timely updates on the latest advancements.

E.0.2 Wikipedia for Material Science

Wikipedia is an open and free resource, allowing users to access a vast amount of information without any cost. It covers fundamental concepts, history, applications, and notable figures in material science, providing a thorough background knowledge base. The content is maintained and updated by a global community, ensuring its timeliness and accuracy.

E.0.3 Material Science Textbook

Textbooks offer systematic, rigorous, and in-depth foundational theories and knowledge in material science. Typically authored by academic and industry experts, the content undergoes stringent review and verification, ensuring high authority and reliability. As primary references for teaching and self-study, their rigor and authority greatly enhance the knowledge base’s service to users.

E.0.4 Material Science Dataset

Datasets provide extensive actual measurement and experimental data, supporting data-driven research and analysis. These datasets undergo strict collection and verification processes, ensuring their authenticity and reliability. They are crucial for validating theoretical models and hypotheses, thereby enhancing the credibility and accuracy of research.

Large datasets are instrumental in training machine learning models, advancing AI applications in material science. The sharing and reuse of data reduce the redundancy of experiments, promoting the sustainability of scientific research.

E.0.5 Material Science Formula

Formulas are fundamental tools for describing and calculating the properties and behaviors of materials, indispensable in material science research and application. Collecting and organizing commonly used formulas allow researchers and engineers to quickly find and use them when needed, reducing search time and increasing efficiency, especially in frequent calculations. Providing standardized formulas and symbols minimizes misunderstandings and errors, improving communication efficiency.

E.0.6 GPT-generated Examples

We utilize GPT-4 to generate examples related to material science by converting multiple-choice questions from high school chemistry into QA format and expanding upon them. This leverages the in-context learning capabilities of large language models (LLMs). These generated examples, verified for accuracy and professionalism, can rapidly produce a large amount of material science-related text as needed, supporting preliminary research and text analysis, thereby enhancing the practicality and diversity of the knowledge base.

These components collectively form a comprehensive, systematic, and practical material science knowledge base, capable of meeting the diverse needs of different users.

• Google Search

The Google Search tool enables the language model to access a vast array of information from the web. By querying search engines through SerpAPI, the tool compiles a selection of results from the first page of Google search results. This allows the model to gather current and relevant information on a wide range of topics, thereby expanding its knowledge base and providing up-to-date answers. The inclusion of this tool ensures that the model can efficiently address queries that require recent or specific web-based information.

• Google Scholar Search

Google Scholar Search is designed to access academic and scholarly articles. It queries Google Scholar to find relevant publications,

1121	papers, and research documents. This tool	cantly enhances the model's ability to perform	1170
1122	is particularly useful for providing accurate	complex tasks that require programming and	1171
1123	and well-supported answers to scientific and	computational skills.	1172
1124	academic inquiries. By leveraging Google		
1125	Scholar, the model can include references to	• Electrochemistry	1173
1126	peer-reviewed articles, enhancing the reliabil-		
1127	ity and credibility of the information it offers.	• Electrical and Thermal Conductivity	1174
		Used to study the relationship between cur-	1175
1128	• Arxiv Search	rent and voltage in chemical reactions, par-	1176
1129	Arxiv Search allows the model to retrieve	ticularly in evaluating battery and electrolyte	1177
1130	the latest preprints and research papers from	performance, analyzing corrosion behavior,	1178
1131	Arxiv.org. This tool is crucial for accessing	and optimizing electroplating processes.	1179
1132	cutting-edge research across various scientific		
1133	fields, particularly in physics, mathematics,	• Corrosion Science	1180
1134	computer science, and quantitative biology.	Used to measure and analyze the electrical	1181
1135	Utilizing Arxiv Search helps the model stay	and thermal conductivity of materials, helping	1182
1136	informed about the most recent advancements	to understand their performance in electronic	1183
1137	and trends in scientific research, which is es-	devices and thermal management systems.	1184
1138	essential for delivering precise and relevant an-		
1139	swers.	• Thermodynamics and Phase Transforma-	1185
		tions	1186
1140	• Wikipedia Search	Used to study the corrosion behavior of metals	1187
1141	Wikipedia Search enables the language model	and alloys in different environments, develop	1188
1142	to quickly access comprehensive and well-	corrosion prevention measures, and extend the	1189
1143	structured information from Wikipedia. As	lifespan of materials.	1190
1144	a widely-used free resource, Wikipedia pro-		
1145	vides valuable background knowledge, his-	• Materials Properties and Unit Cell Calcula-	1191
1146	torical context, and summaries of complex	lations	1192
1147	topics. This tool is instrumental for obtaining	Used to calculate and analyze the crystal struc-	1193
1148	foundational information and ensuring that	ture, mechanical properties, electronic struc-	1194
1149	the model's responses are grounded in widely	ture, and other properties of materials, aiding	1195
1150	accepted knowledge.	in the design and optimization of new materi-	1196
		als. Used to study the thermodynamic prop-	1197
1151	• YouTube Search	erties and phase transformations of materi-	1198
1152	YouTube Search provides the capability to	als, understanding their stability and structural	1199
1153	find educational and explanatory videos on	changes under different temperatures and pres-	1200
1154	YouTube. This tool is especially beneficial	sures	1201
1155	for retrieving visual and auditory content that		
1156	can aid in understanding complex concepts or	• Mechanics of Materials	1202
1157	demonstrating procedures. By incorporating	Used to study the mechanical properties of	1203
1158	YouTube Search, the model can suggest rele-	materials under external forces, such as stress,	1204
1159	vant videos that complement textual explana-	strain, and hardness, helping to evaluate their	1205
1160	tions, thereby enhancing the overall learning	structural performance and durability.	1206
1161	experience for users.		
		• Metallurgy and Alloy Design	1207
1162	• Python REPL	Used to design and optimize the composition	1208
1163	Python REPL (Read-Eval-Print Loop) is a	and processing of metals and alloys, enhanc-	1209
1164	standard tool that provides the language model	ing their mechanical properties and chemical	1210
1165	with a functional Python shell. This tool en-	stability.	1211
1166	ables the model to write and execute Python		
1167	code directly, facilitating a wide range of tasks	• Kinetics and Diffusion	1212
1168	from numerical computations to data analysis	Used to study the reaction rates and diffusion	1213
1169	and AI model training. Python REPL signifi-	behavior in materials, helping to understand	1214
		their behavior and stability in different envi-	1215
		ronments.	1216

1217	• Material Processing and Fabrication
1218	Used to develop and optimize material pro-
1219	cessing techniques such as casting, forging,
1220	and welding, improving material performance
1221	and manufacturing efficiency.
1222	• Unit Conversion and General Utilities
1223	Provides various unit conversion tools and
1224	general calculation functions, facilitating use
1225	in research and engineering.
1226	F Examples of Inductive Tool
1227	Construction
1228	See figure 5.

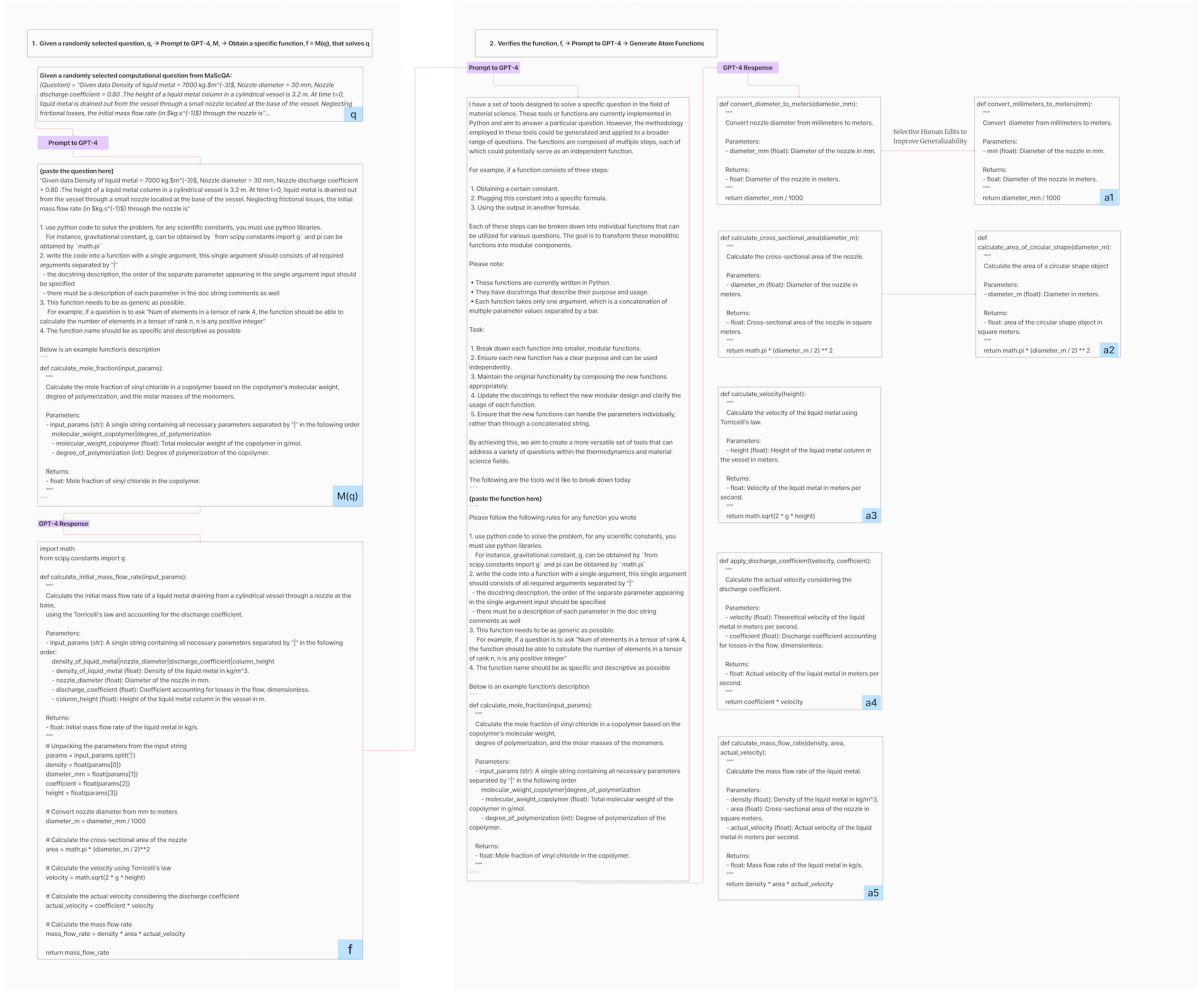


Figure 5: An example of inductive tool construction